SENTINEL

Version 2 — Feature Architecture

Six planned additions to the SENTINEL command and control simulation, covering three frontend capabilities and three backend/AI upgrades.

Document	SENTINEL v2 Architecture
Version	2.0 — Draft
Status	Planning phase — pre-implementation
Date	March 2026
Builds on	SENTINEL Frontend Architecture (v1, March 2026)

1. Overview

This document records the architecture decisions for the six features selected for SENTINEL v2. It follows the same conventions as the v1 Frontend Architecture document: decisions and rationale only, no implementation code, no design specs until finalised.

The six features are grouped into two tracks that can be developed in parallel.

Track	Feature	Primary change	Priority
Frontend	Movement trails	Zustand position history + MapView LineString layer	1
Frontend	Threat heat map overlay	Mapbox native heatmap layer, toggle UI on MapView	2
Frontend	ROE rule editor	Condition builder UI, Zustand rule store, entity evaluation	3
Backend	Streaming LLM explainer	FastAPI SSE route, Anthropic SDK stream(), two-phase frontend render	4
Backend	Anomaly pre-filter model	Isolation Forest alongside RandomForest in model.py	5
Backend	Group behaviour tracking	group_analysis() in loop, new Entity fields, /api/groups route	6

Dependency note

The six features are independent of each other — none requires another to be completed first.

Frontend and backend tracks can proceed in parallel. The recommended priority order above reflects

complexity and incremental value, not sequencing constraints.

2. Frontend — Movement Trails

2.1 Problem statement

Entities are currently represented as static dots on the map. When an entity moves, the previous position disappears with no indication of where the entity came from, how fast it is travelling, or whether it is converging on a friendly unit. Trails address this by retaining a rolling position history per entity and rendering it as a fading line on the map.

Anduril analogy	EagleEye's persistent track history is one of its defining capabilities. An entity that has been circling a location for fifteen minutes looks entirely different from one that just arrived. Trails are what convert dots into behaviour.

2.2 Architecture decisions

Position history — Zustand store

A new slice is added to the Zustand store: `positionHistory: Record<string, Array<{lat: number, lon: number}>>`.

Each time `setEntities()` is called (every 2 seconds via TanStack Query), the store appends the current lat/lon of every entity to its history array. The array is capped at a configurable maximum length (default 15 entries = 30 seconds of history). Entries beyond the cap are dropped from the head of the array.

Rendering — MapView LineString layer

A second GeoJSON source, `"entity-trails"`, is added to MapView alongside the existing entity circles source. It is populated from `positionHistory` on every render. Each entity with two or more history positions contributes one GeoJSON Feature of type LineString.

The layer is rendered as a `line` layer. Line colour matches the entity type colour token. Line opacity is low (0.4) for non-selected entities and higher (0.8) for the selected entity, so the operator's focus entity stands out without the map becoming noisy.

Design decisions

Decision	Chosen approach	Rationale
Trail length	15 ticks (30 seconds)	Long enough to show a patrol route; short enough that the map stays readable with 20 entities
Visibility scope	All entities, non-selected at reduced opacity	Allows operators to spot convergence between non-selected entities without requiring a click

Decision	Chosen approach	Rationale
History storage	Frontend only — Zustand store	No backend changes required. History resets on page refresh, which is acceptable for a live simulation context
Opacity gradient	Uniform per entity type	Mapbox line-gradient requires a GeoJSON LineString feature with line-progress, adding complexity for minimal gain at this scale

2.3 Files touched

File	Change type	Status
frontend/src/store.ts	Add positionHistory slice and appendPosition action	MODIFIED
frontend/src/components/MapView.tsx	Add entity-trails GeoJSON source and line layer	MODIFIED
frontend/src/App.tsx	Call appendPosition on each entity update	MODIFIED

3. Frontend — Threat Heat Map Overlay

3.1 Problem statement

The MapView shows entity positions as discrete dots. There is no way to see the spatial distribution of threats across the operational area — which zones are hotspots, and whether the threat density is shifting over time. A heat map overlay answers this at a glance.

Palantir analogy	Gotham's pattern-of-life heat maps are built by aggregating geolocations over time. The same principle applies here: a density-weighted spatial layer that accumulates as the scenario progresses.

3.2 Two-layer design

Two distinct heat map layers are defined, selectable via a toggle control on the MapView:

Layer	Data source	Update frequency
Live density	Current entity positions, weighted by classifier threat_score. Hostiles and high-scoring ambiguous entities appear as hot zones.	Every 2 seconds — same as entity poll
Engagement history	Lat/lon of entities at the time an operator issued an Approve decision. Requires the backend to log position at decision time. Accumulates over the scenario.	Appended on each Approve decision. Static until next decision.

3.3 Architecture decisions

Live density layer

Mapbox GL JS provides a native `heatmap` layer type. It accepts a GeoJSON FeatureCollection where each Feature carries a `weight` property in [0, 1]. The weight is set to `threat_score` from the classifier, so entities with higher scores generate brighter heat zones.

This layer requires no backend changes. The data is already present in the entity poll response. The GeoJSON source is built client-side from `entities` in the Zustand store.

Engagement history layer

The `/api/entities/{id}/decide` route already receives the operator's decision. A lat/lon field will be added to the `decision_log` entry at decision time. A new endpoint, `GET /api/engagement-history`, returns all historical approve decisions with their coordinates.

This layer accumulates within a session. It does not persist across server restarts in v2 — that is deferred to the threat history persistence work planned separately.

Toggle control

A small control group is added to the top-right corner of MapView with three mode options: Dots only (default), Dots + Live heat, Dots + Engagement history. The selection is stored in local component state — no Zustand change required.

3.4 Files touched

File	Change type	Status
frontend/src/components/MapView.tsx	Add heatmap source, heatmap layer, toggle control	MODIFIED
backend/api/routes.py	Add lat/lon to decision_log entry; add /api/engagement-history route	MODIFIED

4. Frontend — Rules of Engagement (ROE) Rule Editor

4.1 Problem statement

The operator currently has no mechanism to define policy constraints on the classification system. The only way to change how SENTINEL evaluates threats is to modify Python source code. The ROE rule editor gives the operator a UI to define named conditions — signal combinations, proximity thresholds, speed ranges — that automatically modify how entities are presented in the queue and dossier.

Design principle

This is the concrete expression of the human-in-the-loop thesis: the operator sets policy, the AI works within it.
The rule editor makes the oversight relationship visible and interactive, not just implied.

4.2 Rule data model

A rule is a named object with an array of conditions and a resulting action. Conditions are AND-joined within a rule; multiple rules are OR-joined (any matching rule fires).

Field	Type	Description
id	string (UUID)	Unique identifier. Generated client-side.
name	string	Operator-defined label shown in the rule list.
conditions	Condition[]	Array of conditions, all of which must be true for the rule to fire.
action	RuleAction enum	What happens when the rule fires. See action types below.
enabled	boolean	Rules can be toggled without deletion. Default true.

Condition fields

Field	Type	Values / notes
field	string	One of the 13 entity feature names (speed, has_encrypted_comms, distance_to_hostile_zone, etc.)
operator	string	">" | "<" | "=" | ">=" | "<=" for numeric fields; "true" | "false" for boolean fields
value	number | boolean	The threshold value. Numeric for speed, distance, deviation; boolean for has_ prefixed features.

Rule action types

Action	Effect
FLAG_PRIORITY	Entity is pinned to the top of the TargetQueue regardless of sort order. A "ROE" badge appears on the queue row.
WARN_IN_DOSSIER	A rule-match warning banner appears in the DossierPanel header when the entity is opened. Lists the matching rule name.
REQUIRE_EXPLANATION	The Approve button in ActionBar is disabled until the operator has triggered the LLM Explain flow. Forces deliberation before action.
AUTO_DENY	Entity is automatically denied on spawn. Intended for protective rules: any entity carrying a civilian_signal within 0.5 km of a friendly is auto-denied.

4.3 Architecture decisions

Client-side only (v2)

Rules are stored in a new Zustand slice (`rules: ROERule[]`) and evaluated client-side on every entity render. This means rules do not affect the simulation or backend scoring — they only change how entities are presented. Backend enforcement is deferred to v3.

Rationale: client-side evaluation keeps v2 scope contained, avoids API changes, and still delivers the full operator experience. The data model is designed to be portable to a backend rule engine without structural changes.

Rule evaluation timing

Rules are evaluated in the TargetQueue and DossierPanel components, not in a central selector. Each component reads `rules` from the store and evaluates them against the entity in scope. This avoids a new derived state layer and keeps the evaluation logic co-located with the UI that acts on it.

UI structure

The ROE editor is accessed via a new "ROE" button in the Header, which opens a slide-over panel (not a modal — the map remains visible behind it). The panel has two sections: a rule list on the left and a condition builder on the right. No route change is required.

4.4 Files touched

File	Change type	Status
frontend/src/store.ts	Add rules slice: ROERule[], addRule, removeRule, toggleRule actions	MODIFIED
frontend/src/components/ROEEditor.tsx	New component: rule list + condition builder slide-over panel	NEW

File	Change type	Status
frontend/src/components/Header.tsx	Add ROE button that opens editor panel	MODIFIED
frontend/src/components/TargetQueue.tsx	Apply FLAG_PRIORITY sorting and ROE badge	MODIFIED
frontend/src/components/DossierPanel.tsx	Apply WARN_IN_DOSSIER banner and REQUIRE_EXPLANATION gate	MODIFIED
frontend/src/components/ActionBar.tsx	Disable Approve when REQUIRE_EXPLANATION rule matches and explain not yet run	MODIFIED
frontend/src/types.ts	Add ROERule, Condition, RuleAction types	MODIFIED

5. Backend — Streaming LLM Explainer

5.1 Problem statement

The current explain endpoint blocks until Claude returns a complete response (~2–4 seconds), then delivers the full JSON object at once. The operator sees a static "Analysing..." indicator for the entire wait. Streaming eliminates the perception of latency by delivering reasoning text token by token as Claude produces it.

UX rationale The perceived latency of streaming is dramatically lower than blocking even when total response time is identical.
Reading Claude's reasoning as it is constructed also reinforces the deliberative quality of the AI — the operator
sees the hedging and evidence-weighing happen in real time rather than receiving a finished verdict.

5.2 Architecture: two-phase streaming

The current explain response is a structured JSON object with seven fields (summary, reasoning, agrees_with_classifier, confidence_assessment, recommended_action, recommended_action_rationale, civilian_risk, civilian_risk_rationale). Streaming raw tokens would break JSON structure. The solution is a two-phase response:

Phase	Content	Delivery method
1 — Reasoning	The reasoning array items and summary, delivered as a continuous prose stream	SSE text/delta events — one event per token chunk from Claude
2 — Structured	The remaining structured fields: recommended_action, civilian_risk, agrees_with_classifier, confidence_assessment	Single SSE data event at stream end, containing a JSON object with the structured fields only

5.3 Backend changes

New route: GET /api/entities/{id}/explain/stream

A new route is added alongside the existing `GET /explain` route (which is kept for backward compatibility). The new route returns a `StreamingResponse` with media type `text/event-stream`.

The Anthropic Python SDK's `client.messages.stream()` context manager is used. It yields text deltas as they arrive from the Claude API. The route wraps this in an async generator that emits SSE-formatted events.

Prompt changes

The prompt is split into two calls. The first call streams the reasoning narrative. The second call (non-streaming, fired after the stream closes) retrieves the structured fields in JSON. This is more reliable than attempting to parse structured JSON from a streaming response.

5.4 Frontend changes

LLMExplainer.tsx

The `api.getExplain()` fetch is replaced with an `EventSource` connection to the new stream route. Two state variables manage the two phases:

- streamingText — string — accumulates phase 1 token chunks, rendered as prose with a blinking cursor
- structuredResult — object | null — null until phase 2 data event arrives

When the structured result arrives, the blinking cursor is removed and the action badge, confidence bar, and civilian risk indicator animate in. This creates the effect of Claude reaching a conclusion after reasoning through the evidence.

5.5 Files touched

File	Change type	Status
backend/api/routes.py	Add GET /api/entities/{id}/explain/stream route	NEW
backend/api/explainer.py	Add stream_explain_entity() function using client.messages.stream()	MODIFIED
frontend/src/components/LLMExplainer.tsx	Replace fetch with EventSource; add two-phase render	MODIFIED
frontend/src/api.ts	Add streamExplain() function returning EventSource	MODIFIED

6. Backend — Anomaly Pre-Filter Model

6.1 Problem statement

The RandomForest classifier is trained on a fixed set of entity profiles. It is confident — it always assigns the entity to the nearest trained category. This means it will confidently misclassify a genuinely novel threat as ambiguous or civilian if that threat's feature combination falls between trained categories. An unsupervised anomaly detector catches entities that are statistically unusual regardless of label.

What this catches that the RandomForest misses	A civilian entity with an encrypted_comms signal would confuse the RandomForest — civilians never have that signal, but the RF might still score it low if other civilian features dominate. An Isolation Forest trained on civilian and friendly profiles only would immediately flag this as anomalous, regardless of its overall threat score.

6.2 Algorithm choice: Isolation Forest

Isolation Forest (scikit-learn `IsolationForest`) is chosen for the following reasons:

- Already in scikit-learn — zero new dependencies
- Trains on the same 13-feature vectors already extracted by extract_features()
- decision_function() returns a continuous anomaly score — negative values indicate anomalies
- Works well on small tabular datasets (2,000 training samples)
- Training time is negligible alongside the RandomForest

6.3 Training approach

The Isolation Forest is trained on civilian and friendly entity feature vectors only — the "normal" population. Hostile and ambiguous entities are excluded from training. This means the model learns what normal looks like, and anything that deviates strongly registers as anomalous.

The `contamination` parameter is set to 0.05 (5% expected anomaly rate in the normal population). This is tunable after observing false positive rates in practice.

Training is called from `train_classifier()` in `model.py` — the same function that trains the RandomForest on startup. Both models train in the same pass over the synthetic data.

6.4 Integration points

/api/entities/{id}/score response

Two new fields are added to the score response:

•
•

Claude prompt enrichment

When `anomaly_flag` is true, a new section is appended to the `_build_prompt()` output: "ANOMALY DETECTION: This entity's feature combination is statistically unusual compared to the normal population baseline. Consider this independently of the classifier's label." Claude is instructed to reason about the anomaly flag as a separate signal rather than allowing it to override the classifier.

DossierPanel — ClassifierResult.tsx

When `anomaly_flag` is true, an ANOMALY badge is rendered in ClassifierResult, styled in amber, positioned above the threat score bar. This gives the operator a visual signal before they request the LLM explanation.

6.5 Files touched

File	Change type	Status
backend/classifier/model.py	Add IsolationForest training, AnomalyClassifier wrapper class, anomaly_score() method	MODIFIED
backend/api/routes.py	Add anomaly_score and anomaly_flag to /score response	MODIFIED
backend/api/explainer.py	Inject anomaly flag context into _build_prompt() when flag is true	MODIFIED
frontend/src/components/ClassifierResult.tsx	Render ANOMALY badge when anomaly_flag is true in score data	MODIFIED

7. Backend — Group Behaviour and Relationship Tracking

7.1 Problem statement

The current system assesses every entity in isolation. Claude reasons about a single entity's signals without any awareness of nearby entities, their movements, or their relationship to each other. Real threat assessment is fundamentally relational — a single entity carrying encrypted comms is suspicious; three entities all switching to encrypted comms within the same 30-second window at the same location is a qualitatively different threat signal.

Palantir analogy	Gotham's graph engine is the source of its analytical power — not individual entity scoring, but the relationships between entities over time. Group behaviour tracking adds the foundational layer that makes relational reasoning possible.

7.2 Behaviours detected

Behaviour	Detection logic	Threat relevance
Formation movement	Two or more entities maintaining a consistent pairwise distance (< 0.5 km) and similar heading (within 30°) across three or more consecutive ticks	Coordinated approach — significantly raises threat assessment of ambiguous entities
Civilian dispersal	A cluster of civilian entities (≥ 3, within 0.3 km of each other) that spreads to > 1 km separation within 4 ticks	Indirect threat signal — indicates something caused local population to scatter
Encirclement	A friendly unit is within the convex hull of three or more hostile or ambiguous entities, computed per tick	Direct threat to friendly — prioritises any encircling ambiguous entity in the queue
Signal correlation	Two or more entities showing the same signal type (e.g., encrypted_comms) within a 30-second window and within 2 km of each other	Coordination indicator — temporal correlation of signals is a strong adversarial marker

7.3 Architecture

group_analysis() function — simulation loop

A new function `group_analysis(entities: list[Entity], sim_time: float) -> dict[str, GroupAssignment]` is added to the simulation loop. It is called after the movement update step each tick, before cleanup.

The function computes pairwise distances using the existing lat/lon conversion constants from `engine.py`. Clustering uses a simple radius-based approach (no DBSCAN dependency) — if two entities are within the proximity threshold, they are placed in the same group. Groups are

assigned stable IDs using a hash of their sorted member entity IDs, so a group's ID persists across ticks as long as its membership is stable.

New fields on Entity dataclass

Field	Type	Description
group_id	Optional[str]	Stable group identifier. None if entity is not in a group.
group_behaviour	Optional[str]	Detected behaviour name. None if no behaviour detected for this group.
group_size	int	Number of entities in the group, including this one. 1 if not grouped.

These fields are propagated through `EntityOut` and appear in the existing `/api/entities` response automatically.

New route: GET /api/groups

A dedicated groups endpoint returns the current group assignments in a format optimised for the frontend map overlay and future analytics use.

Response shape: `{ groups: [{ group_id, entity_ids, behaviour, centroid: {lat, lon}, threat_level }] }`.

The `centroid` field is the mean lat/lon of all member entities, useful for displaying a group-level marker or bounding box on the map. The `threat_level` field ("low" | "elevated" | "high") is derived from the composition of the group: groups containing confirmed hostiles are "high"; groups of ambiguous entities exhibiting formation movement are "elevated".

Claude prompt enrichment

The most significant impact of group tracking is on Claude's reasoning quality. When `explain_entity()` is called for an entity that belongs to a group, a "GROUP CONTEXT" section is appended to the prompt:

- Group size and composition (e.g., "2 ambiguous, 1 unclassified")
- Detected behaviour name and a plain-language description
- Distance to nearest friendly unit from the group centroid
- How long the group has been together (in ticks)

Claude is instructed to treat the group context as a separate line of evidence and to weight it accordingly — a solo entity with encrypted comms is handled differently from the same entity when it is the third member of a formation moving toward a friendly.

7.4 Files touched

File	Change type	Status
backend/simulation/loop.py	Add group_analysis() call in simulation tick; import GroupAssignment	MODIFIED
backend/simulation/groups.py	New module: group_analysis(), GroupAssignment dataclass, behaviour detectors	NEW
backend/simulation/engine.py	Add group_id, group_behaviour, group_size fields to Entity dataclass	MODIFIED
backend/api/routes.py	Add GET /api/groups route; propagate group fields through EntityOut	MODIFIED
backend/api/explainer.py	Inject group context into _build_prompt() when entity is grouped	MODIFIED

8. Recommended Implementation Order

The six features are independent of each other and can be built in any order or in parallel across tracks. The sequence below maximises incremental visible progress:

#	Feature	Rationale for this position	Definition of done
1	Movement trails	Pure frontend, no backend touch, lowest complexity. Fast first win.	Fading lines appear on map behind moving entities. Selected entity trail is brighter.
2	Streaming explainer	Highest-impact UX change. Every subsequent demo session benefits from it.	Explain text streams token by token. Structured fields animate in at stream end.
3	Anomaly pre-filter	Self-contained in model.py. Adds a new AI signal visible immediately in the dossier.	ANOMALY badge appears in ClassifierResult. Anomaly context visible in Claude output.
4	Heat map overlay	Live density layer is pure frontend. Engagement history layer needs the route change from step 3 already done.	Toggle control on map switches between dot, live heat, and engagement history views.
5	Group behaviour tracking	Largest backend change. Most impactful upgrade to Claude's reasoning quality.	Group markers visible on map. Group context appears in Claude explanation for grouped entities.
6	ROE rule editor	Most complex frontend feature. Benefits from group tracking being complete so ROE conditions can reference group behaviour.	Operator can create, enable, and delete rules. FLAG_PRIORITY and REQUIRE_EXPLANATION actions visibly affect queue and dossier.

Parallel development note

Frontend track (trails, heat map, ROE) and backend track (streaming, anomaly, groups) share no file overlap.

Both tracks can be developed simultaneously without merge conflicts until ROE rule editor reaches its final step

(group_behaviour as a condition field), which requires the groups.py module to be complete.

SENTINEL v2 Architecture — Internal document — March 2026